



School of Computer Science

COMP47470: Big Data Programming

Practicals in Python

Module Coordinator	Ravi Reddy Manumachu
Email	ravi.manumachu@ucd.ie

Basic Python	1
Functional Programming in Python	3
Object-Oriented Programming in Python	4

Basic Python

1. Write a program that prints the following string to the terminal:
Hello, World!
2. Write a program that uses Euclid's algorithm to find GCD of two numbers:
 - a). Declare a function that takes two arguments, ***a*** and ***b***.

- b). The function must return the GCD of the two numbers
(Note: Recursive call to the function can be used)
3. Write a program that executes the following steps:
- Create a list containing numbers from 1, 2, ..., 100 using a list comprehension
 - Print the sum of the numbers to the terminal
4. Write a program that executes the following steps:
- Declare a function that accepts two lists as arguments
 - The function must return the scalar product of the two lists. The scalar product of two lists is computed as follows:

$$[A_1, A_2, \dots, A_n] \cdot [B_1, B_2, \dots, B_n] = A_1 \cdot B_1 + A_2 \cdot B_2 + \dots + A_n \cdot B_n$$
 - To test the function,
 - Create two lists, each of size 10, filled with pre-determined integers
 - Call the function using the two lists as arguments
 - Print the result to the terminal

5. Use program 3 to find the sum of the squares: $1^2 + 2^2 + 3^2 + \dots + 100^2$

6. Write a program that implements binary search:
- Declare a function that accepts two arguments:
 - List called 'haystack' of sorted integers
 - A item called 'needle', which is an integer
 - The function must use binary search to find the needle
 - If the item is found, it must print the index where the item is found
 - Otherwise, a message "Needle not found" must be printed

To test the function,

- Create a list of 100 random integers
- Sort the list
- Call the function with sorted list and different integral values for the needles.

7. Consider the following list of tuples (author, work):
- ```
IrishLiteraryGiants = [("Wilde","The Ballad of Reading Gaol"), ("Doyle","Paddy Clarke Ha Ha Ha"), ("Beckett","Waiting for Godot"), ("Joyce", "Ulysses"), ("Binchy", "Scarlet Feather"), ("Donaghue","Room")]
```

- Create a dictionary called *ILGDict* containing key:value pairs where key is author and value is the work of the author
- Use dictionary comprehension to create the dictionary from the list of

tuples

c). Declare a function **FindWork** that takes two arguments, one author and the other dictionary

d). Given the author, the function must print the work of the author.

For example:

FindWork("Donaghue",*ILGDict*) must print "Room"

FindWork("Shakespeare",*ILGDict*) must print

**Traceback (most recent call last):**

...

**KeyError: 'Shakespeare'**

## Functional Programming in Python

In this section, you will use functional constructs in Python.

The higher-order functions (HOF), **map** and **filter**, are built-in. To use the HOF **reduce**, you must import it first as follows:

import operator

from functools import reduce

1. Write a program that executes the following steps:
  - Create a list '**lst**' of random integers between 1 and 100. To generate a pseudo-random integer between 1 and 100, use the following:  
**from random import randint**  
**randint(1,100)**
  - Given the '**lst**' as input, output a list of even numbers using the HOF, **filter**
2. Write a program that produces the same output as **Program1** but using a list comprehension.
3. Write a program that produces the same output as **Program1** but using a generator expression.
4. Find the sum of integers  $1+2+\dots+100$  using HOF, **reduce** using a single call to it.
5. Find the sum of integers  $1^2+2^2+\dots+100^2$  using HOF, **reduce** using a single call to it. (Note: You may have to use a list comprehension within the **reduce** call)

6. A triple  $(x,y,z)$  of positive integers is *Pythagorean* if it satisfies the equation  $x^2+y^2=z^2$ . Using a generator expression and HOF, **filter**, generate *Pythagorean* triples where  $(x \leq 10, y \leq 10, z \leq 10)$ .
7. Write a program that converts all the characters in a file to uppercase and prints them as follows:
  - Create a file "foo.txt" with the following contents:
  - Open the file in Python for reading using the Python I/O as follows:  
`fi = open("foo.txt","r+")`
  - Create a generator expression 'g' for the file, which converts the contents of the file to uppercase.
  - From the generator, print the contents to the terminal as follows:  
`print("".join(g))`

## Object-Oriented Programming in Python

In this section, we will write a class for complex numbers. The solution is provided in complex.py.

A complex number is composed of two parts, a real number *re* and an imaginary number *im*, and is represented mathematically by  $re + im * i$ , where *i* is the imaginary part satisfying the equation,  $i^2 = -1$ .

1. Create a file, 'complex.py', which will contain the definitions for the class for complex numbers.
2. The class will have two instance variables, *re* and *im*, which will contain the real and the imag parts. The instance variables will store *re* and *im* as floats. Fill the body of the `__init__` method.

**class** Complex:

**def** `__init__`(**self**, real, imag):

3. Create an instance of class Complex as follows:

```
$ python3 -i complex.py
>>> c = Complex(2.0,3.0)
>>> str(c)
<__main__.Complex object at 0x7fe4ab15e2e8>
>>>
```

As you can see, 'c' is not printed in its mathematical representation, 2.0 +

3.0i.

4. To print a complex number in its mathematical representation, the following special method must be implemented:

```
class Complex:
 def __str__(self):
```

Note that `__str__` must return a string representation of the object.

5. Test your implementation of `__str__` method as follows:

```
$ python3 -i complex.py
>>> c = Complex(2.0,3.0)
>>> str(c)
'2.0 + 3.0i'
```

6. We will use operator overloading now. Make sure you adhere to the fundamental rule of operators, which is that they always return a new object. So, create and return a new instance wherever necessary.
7. Fill the body of the special method `__abs__` that will print the absolute value of the complex number.

```
class Complex:
 def __abs__(self):
```

Use the `'hypot'` function from the `'math'` package to calculate the Euclidean norm of two numbers.

```
import math
math.hypot(self.re, self.im)
```

8. Test your implementation of the `__abs__` method.

```
$ python3 -i complex.py
>>> c = Complex(2.0,3.0)
>>> abs(c)
3.605551275463989
```

9. Fill the body of `__neg__` method that will build a new complex number, which will negate the two components of `self`.

```
class Complex:
 def __neg__(self):
```

10. Test your implementation of the `__neg__` method.

```
$ python3 -i complex.py
>>> c = Complex(2.0,3.0)
>>> str(-c)
'-2.0 + -3.0i'
```

11. Fill the body of `__add__` method that will add two complex numbers.

```
class Complex:
 def __add__(self, other):
```

12. Test your implementation of the `__add__` method.

```
$ python3 -i complex.py
>>> c1 = Complex(2.0,3.0)
>>> c2 = Complex(4.0,7.0)
>>> str(c1+c2)
'6.0 + 10.0i'
>>> c3 = c1+c2
>>> str(c3)
'6.0 + 10.0i'
```

13. Similarly, implement the `__sub__` method and test your implementation as follows:

```
class Complex:
 def __sub__(self, other):
```

```
$ python3 -i complex.py
>>> c1 = Complex(2.0,3.0)
>>> c2 = Complex(4.0,7.0)
>>> str(c1-c2)
'-2.0 + -4.0i'
>>> c3=c1-c2
>>> str(c3)
'-2.0 + -4.0i'
```

14. Implement the `__mul__` method to multiply two complex numbers and test your implementation as follows:

```
class Complex:
 def __mul__(self, other):
```

```
$ python3 -i complex.py
>>> c1 = Complex(2.0,3.0)
>>> c2 = Complex(4.0,7.0)
>>> str(c1*c2)
'-13.0 + 26.0i'
>>> c3=c1*c2
>>> str(c3)
'-13.0 + 26.0i'
```

15. One can multiply a complex number by a scalar as follows:

```
$ python3 -i complex.py
>>> c1 = Complex(2.0,3.0)
>>> c2 = c1 * 4
>>> str(c2)
'8.0 + 12.0i'
```

Implement the `__mul__` method to multiply a complex number by a scalar as follows:

```
class Complex:
 def __mul__(self, scalar):
```

16. However, if you multiply as follows, you get a `TypeError`:

```
$ python3 -i complex.py
>>> c1 = Complex(2.0,3.0)
>>> c2 = 4 * c1
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for *: 'int' and 'Complex'
```

17. To implement this functionality, you must implement what is known as "righthand" special method, `__rmul__`.

```
class Complex:
 def __rmul__(self, scalar):
```

In your implementation, you can reuse the implementation of `__mul__`

(how?).

18. Test your implementation of `__rmul__` as follows:

```
$ python3 -i complex.py
>>> c1 = Complex(2.0,3.0)
>>> c2 = 4 * c1
>>> str(c2)
'8.0 + 12.0i'
```

19. Implement division of two complex numbers using the `__truediv__` method.

```
class Complex:
 def __truediv__(self, other):
```

The division of two complex numbers,  $a+bi$  and  $c+di$ , is calculated as follows:

$$(ac - bd) / (c^2 + d^2) + (bc - ad) / (c^2 + d^2)i$$

20. Test your implementation as follows:

```
$ python3 -i complex.py
>>> c1 = Complex(2.0,3.0)
>>> c2 = Complex(1.0,2.0)
>>> c3 = c1/c2
>>> str(c3)
'3.5777087639996634 + -0.4472135954999579i'
```

21. Implement `__eq__` to check the equality of two complex numbers.

```
class Complex:
 def __eq__(self, other):
```

Two complex numbers are equal if their real parts are equal, and their imaginary parts are equal.

Note that a complex number is composed of two floats and equality test for two floating-point numbers can fail.

So, use this method only if you are composing complex numbers from integers.

22. Test your implementation as follows:

```
$ python3 -i complex.py
```

```
>>> c1 = Complex(2.0,3.0)
>>> c2 = Complex(2.0,3.0)
>>> c1 == c2
True
>>> c3 = Complex(1.0,3.0)
>>> c2 == c3
False
```

23. Implement augmented assignment operators that allow you to add, subtract, multiply complex numbers in-place as follows:

```
class Complex:
 def __iadd__(self, other):
 def __isub__(self, other):
 def __imul__(self, other):
```

Reuse implementations for `__add__`, `__sub__`, `__mul__`.

24. Test your implementations as follows:

```
$ python3 -i complex.py
>>> c1 = Complex(2.0,3.0)
>>> c1 += Complex(1.0,2.0)
>>> str(c1)
'3.0 + 5.0i'
>>> c2 = Complex(4.0,5.0)
>>> c2 -= Complex(2.0,3.0)
>>> str(c2)
'2.0 + 2.0i'
>>> c3 = Complex(6.0,8.0)
>>> c3 *= Complex(2.0,3.0)
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
 File "complex.py", line 35, in __imul__
 return self * other
 File "complex.py", line 38, in __mul__
 return Complex(self.re * scalar, self.im * scalar)
 File "complex.py", line 6, in __init__
 self.re = float(real)
TypeError: float() argument must be a string or a number, not 'Complex'
>>>
```

25. Can you fix this error?

Hint: Check if the input is 'Complex' using *isinstance* function and multiply accordingly.

Test your implementation as follows:

```
$ python3 -i complex.py
>>> c3 = Complex(6.0,8.0)
>>> c3 *= Complex(2.0,3.0)
>>> str(c3)
'-12.0 + 34.0i'
```

26. Implement the exponential of a complex number.

**def** exp(z):

An exponential of a complex number 'z = a+bi' is determined as follows:

$$e^{a+bi} = e^a * e^{bi}$$

The Euler's formula for  $e^{bi} = \cos(b) + i\sin(b)$

Use `math.cos` and `math.sin` functions to determine the exponential.